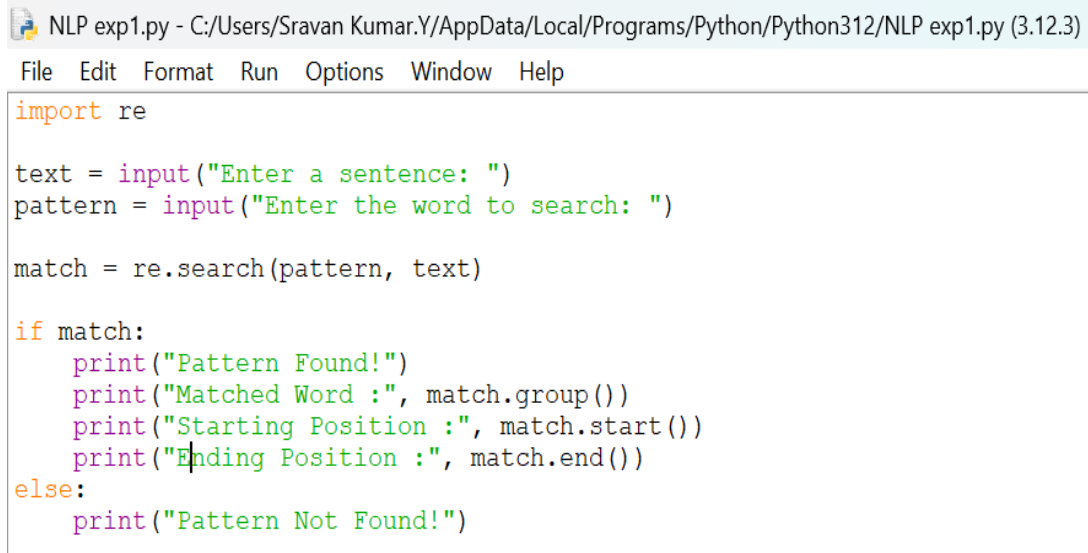


DSA0306-Natural Language Processing

List of Lab Program

1.The writing program demonstrates how to use regular expressions in Python to match and search for patterns in text.

Code:



```
NLP exp1.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP exp1.py (3.12.3)
File Edit Format Run Options Window Help

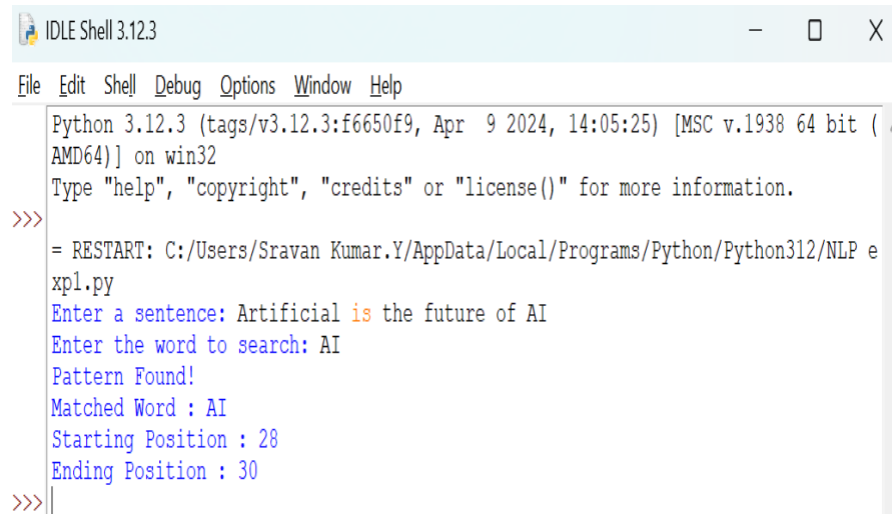
import re

text = input("Enter a sentence: ")
pattern = input("Enter the word to search: ")

match = re.search(pattern, text)

if match:
    print("Pattern Found!")
    print("Matched Word :", match.group())
    print("Starting Position :", match.start())
    print("Ending Position :", match.end())
else:
    print("Pattern Not Found!")
```

Output:



```
IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help

Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP exp1.py
Enter a sentence: Artificial is the future of AI
Enter the word to search: AI
Pattern Found!
Matched Word : AI
Starting Position : 28
Ending Position : 30
>>>
```

2. Implement a basic finite state automaton that recognizes a specific language or pattern. In this example, we'll create a simple automaton to match strings ending with 'ab' using python.

Code:

```
NLP Ex2.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP Ex...
File Edit Format Run Options Window Help
def check_string(string):
    state = 0

    for ch in string:
        if state == 0:
            if ch == 'a':
                state = 1
            else:
                state = 0

        elif state == 1:
            if ch == 'b':
                state = 2
            elif ch == 'a':
                state = 1
            else:
                state = 0

        elif state == 2:
            if ch == 'a':
                state = 1
            else:
                state = 0

    if state == 2:
        print("Accepted (String ends with 'ab')")
    else:
        print("Rejected")

string = input("Enter a string: ")
check_string(string)
|
```

Output:

```
>>> |
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP E
x2.py
Enter a string: aaab
Accepted (String ends with 'ab')
```

3. Write program demonstrates how to perform morphological analysis using the NLTK library in Python.

Code:

```
NLP Ex3.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP Ex...
File Edit Format Run Options Window Help
import nltk
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer

nltk.download('wordnet')

word = input("Enter a word: ")

ps = PorterStemmer()
lm = WordNetLemmatizer()

print("Original Word :", word)
print("Stemmed Word :", ps.stem(word))
print("Lemmatized Word :", lm.lemmatize(word))
```

Output:

```
>>> = RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP E
x3.py
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP E
x3.py
[nltk_data] Downloading package wordnet to C:\Users\Sravan
[nltk_data] Kumar.Y\AppData\Roaming\nltk_data...
Enter a word: running
Original Word : running
Stemmed Word : run
Lemmatized Word : running
```

4. Implement a finite-state machine for morphological parsing. In this example, we'll create a simple machine to generate plural forms of English nouns using python.

Code:

```
NLP Exp4.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP E...
File Edit Format Run Options Window Help
# Finite State Machine for Morphological Parsing
# Generates plural forms of English nouns

def generate_plural(noun):
    # State q0 (Start State)
    if noun.endswith(("s", "x", "z", "ch", "sh")):
        # Transition to q1
        plural = noun + "es"

    elif noun.endswith("y") and len(noun) > 1 and noun[-2].lower() not in "aeiou":
        # Transition to q2
        plural = noun[:-1] + "ies"

    else:
        # Transition to q3
        plural = noun + "s"

    return plural

# Test words
words = [
    "cat", "dog", "bus", "box",
    "church", "dish", "baby",
    "city", "toy", "book"
]

print("Singular\tPlural")
print("-----")

for word in words:
    print(f"{word:10}\t{generate_plural(word)}")
```

Output:

```
>>> = RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP E
xp4.py
Singular      Plural
-----
cat           cats
dog           dogs
bus           buses
box           boxes
church        churches
dish          dishes
baby          babies
city          cities
toy           toys
book          books
>>>
```

5. Use the Porter Stemmer algorithm to perform word stemming on a list of words using python libraries.

Code:

```
NLP Exp5.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP E...
File Edit Format Run Options Window Help
import nltk
from nltk.stem import PorterStemmer

# Create Porter Stemmer object
stemmer = PorterStemmer()

# List of words
words = [
    "running",
    "playing",
    "studies",
    "studying",
    "connected",
    "connection",
    "happiness",
    "flying",
    "wolves",
    "cars"
]

print("Original Word\t\tStemmed Word")
print("-----")

# Perform stemming
for word in words:
    stem = stemmer.stem(word)
    print(f"{word:20}{stem}")
```

Output:

```
>>> = RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP E
xp5.py
Original Word          Stemmed Word
-----
running               run
playing              play
studies              studi
studying             studi
connected            connect
connection           connect
happiness            happi
flying               fli
wolves              wolv
cars                car
>>>
```

6. Implement a basic N-gram model for text generation. For example, generate text using a bigram model using python.

Morphological Analysis using NLTK (Without Porter Stemmer)

Code:

```
NLP Exp6.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP E...
File Edit Format Run Options Window Help
from nltk.util import bigrams

text = "Artificial Intelligence is transforming the world"

words = text.split()

bigram_list = list(bigrams(words))

print("Original Sentence:")
print(text)

print("\nGenerated Bigrams:")

for bg in bigram_list:
    print(bg)
```

Output:

```
>>> = RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP E
xp6.py
Original Sentence:
Artificial Intelligence is transforming the world

Generated Bigrams:
('Artificial', 'Intelligence')
('Intelligence', 'is')
('is', 'transforming')
('transforming', 'the')
('the', 'world')
>>> |
```